

Sistema de Simulación de Parque de Diversiones

Programación Concurrente 2025

Dante R. Giuliano | FAI - 2062

March 16, 2026

Contents

0.1	Objetivos del Sistema	3
0.2	Tecnologías Utilizadas	3
1	Arquitectura General del Sistema	4
1.1	Diagrama de Arquitectura de Alto Nivel	4
1.2	Patrón Arquitectónico	4
2	Componentes Principales	5
2.1	Main y Simulación	5
2.1.1	Main.java	5
2.1.2	Simulacion.java	6
2.2	Sistema de Control Temporal	7
2.2.1	Reloj.java	7
2.3	Sistema de Ingreso	8
2.3.1	Molinete.java	8
2.3.2	Parque.java	8
2.4	Visitante y Sistema de Fichas	10
2.4.1	Visitante.java	10
2.4.2	Billetera.java	10
2.4.3	Ticketera.java	10
3	Atracciones Mecánicas	11
3.1	Montaña Rusa	11
3.2	Autitos Chocadores	11
4	Atracciones Complejas	13
4.1	Carrera de Gomones	13
4.1.1	Flujo Completo de Participación	14
5	Área de Premios: Diseño y Concurrencia	15
5.1	Responsabilidad del Componente	15
5.2	Diagrama de Clases Simplificado (UML)	15
5.3	Catálogo Inicial de Premios	15
5.4	Modelo de Concurrencia y Sincronización	16
5.5	Lógica de programa	16
6	Comedor	17
6.1	Descripción General	17
6.2	Clases Principales	17
6.3	Flujo de Ejecución Detallado	17
6.4	Mecanismos de Concurrencia Utilizados	18
7	Teatro	19
7.1	Descripción General	19
7.2	Clases Principales	19
7.3	Flujo del Sistema	19
7.3.1	Visitante	19
7.3.2	Asistente	19

7.4	Mecanismos de Sincronización Utilizados	20
7.5	Estados del Teatro	20
8	Realidad Virtual	21
8.1	Descripción General	21
8.2	Clases Principales	21
8.3	Variables y Mecanismos de Sincronización	21
8.4	Flujo de Interacción	22
8.5	Diagrama de Flujo de Interacción	23
8.6	Observaciones	23

1 Introducción

1.1 Objetivos del Sistema

- Simular el funcionamiento concurrente de un parque de diversiones.
- Gestionar el acceso controlado a recursos compartidos (atracciones).
- Prevenir condiciones de carrera y deadlocks.
- Implementar políticas de fairness en el acceso a recursos.
- Generar logs detallados para análisis de comportamiento.

1.2 Tecnologías Utilizadas

- **Lenguaje:** Java 11+
- **Mecanismos de Sincronización:** Semaphores, Locks, Conditions, CyclicBarrier, Exchanger
- **Estructuras de Datos Concurrentes:** BlockingQueue, AtomicInteger, AtomicLong
- **Visualización:** HTML/JavaScript para análisis de logs

2 Arquitectura General del Sistema

2.1 Diagrama de Arquitectura de Alto Nivel

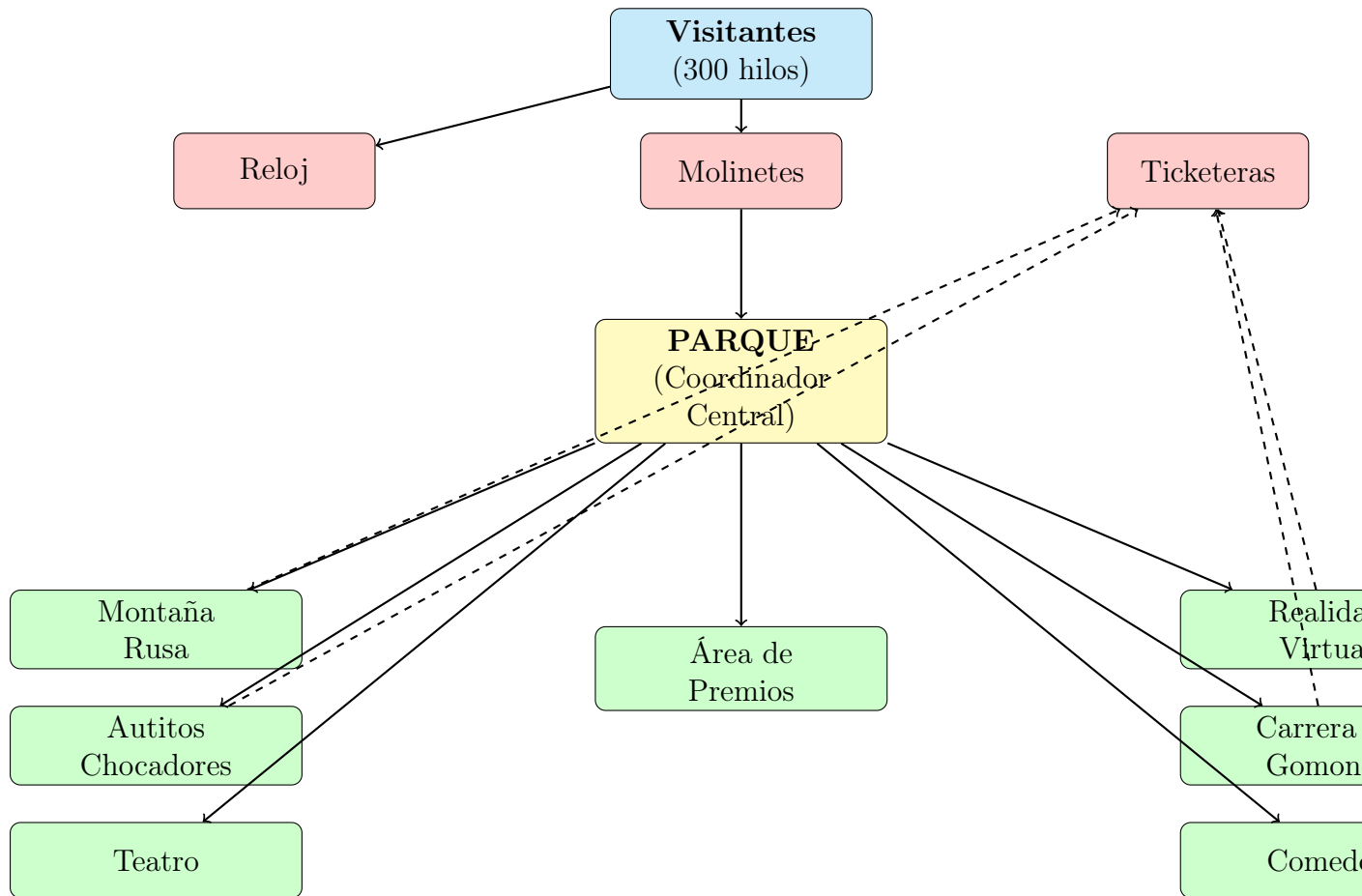


Figure 1: Arquitectura General del Sistema

2.2 Patrón Arquitectónico

El sistema implementa una **arquitectura basada en componentes** con los siguientes principios:

- **Alta cohesión:** Cada componente tiene una responsabilidad única y bien definida.
- **Bajo acoplamiento:** Los componentes interactúan a través de interfaces claras.
- **Separación de responsabilidades:** La lógica del dominio está separada de los mecanismos de sincronización.
- **Patrón Productor-Consumidor:** Utilizado en múltiples atracciones.
- **Patrón Coordinador:** El Parque actúa como coordinador central.

3 Componentes Principales

3.1 Main y Simulación

3.1.1 Main.java

Responsabilidad: Punto de entrada del sistema. Inicializa y configura la simulación.

Parámetros de configuración:

- APERTURA: Tiempo en milisegundos que el parque permanece abierto (12000 ms).
- CIERRE: Tiempo en milisegundos que el parque permanece cerrado (6000 ms).
- VISITANTES: Cantidad de hilos de visitantes a crear (300).
- MOLINETES: Cantidad de molinetes de entrada (3).

Flujo de ejecución:

1. Activa/desactiva generación de logs.
2. Inicia el hilo del Reloj.
3. Crea la instancia del Parque.
4. Genera los visitantes mediante Simulación.
5. Inicia todos los hilos de visitantes.

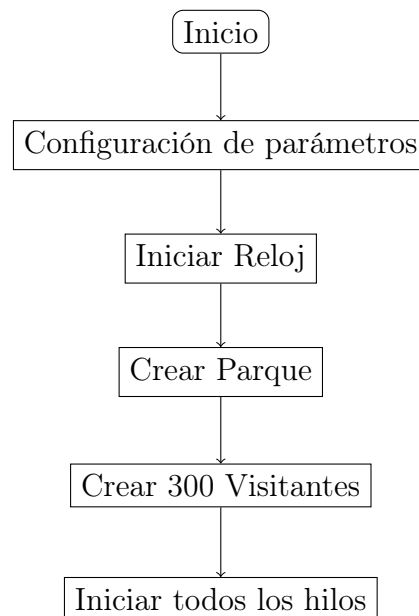


Figure 2: Flujo de inicialización del sistema

3.1.2 Simulacion.java

Responsabilidad: Fábrica para creación de visitantes con IDs únicos.

Características:

- Utiliza `AtomicLong` para generar IDs thread-safe.
- Crea instancias de `Visitante` con `BigInteger` como identificador.
- Retorna lista de hilos configurados pero no iniciados.

Mecanismo de sincronización:

```
1 private static final AtomicLong idGenerator = new AtomicLong(0);  
2 long uniqueId = idGenerator.incrementAndGet();
```

3.2 Sistema de Control Temporal

3.2.1 Reloj.java

Responsabilidad: Gestionar ciclos de apertura/cierre del parque.

Diagrama de estados:

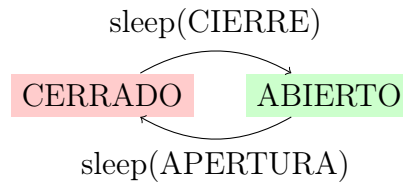


Figure 3: Estados del Reloj

Características técnicas:

- Variable estática `abierto`: Estado global accesible desde cualquier componente.
- Hilo daemon que corre continuamente.
- Método estático `operativo()`: Consulta no bloqueante del estado.
- Genera logs en cada cambio de estado.

Casos de uso:

- Molinetes verifican el estado antes de permitir ingreso.
- Visitantes consultan para saber si pueden continuar en atracciones.
- Atracciones pueden rechazar nuevos ingresos si el parque está cerrado.

3.3 Sistema de Ingreso

3.3.1 Molinete.java

Responsabilidad: Controlar ingreso de visitantes al parque con exclusión mutua.

Mecanismos de sincronización:

- **Semaphore(1, true):** Garantiza exclusión mutua con fairness.
- **BigInteger contador:** Generador de tickets protegido por la sección crítica.
- **Bloque finally:** Asegura liberación del semáforo incluso con excepciones.

Prevención de condiciones de carrera:

```
1 // Double-check del estado del parque
2 if (!Reloj.operativo()) return false;
3 try {
4     semaforo.acquire();
5     // Verificar nuevamente dentro de la sección crítica
6     if (!Reloj.operativo()) return false;
7     // ...
8 } finally {
9     semaforo.release();
10 }
```

3.3.2 Parque.java

Responsabilidad: Coordinador central y fachada del sistema.

Recursos gestionados:

- Lista de molinetes.
- Semáforo de entrada (entradaParque).
- Instancias de todas las atracciones.
- Generador de números aleatorios para decisiones probabilísticas.

Método mapa(): Bucle principal del visitante

```
1 while (Reloj.operativo()) {
2     if (rng(20)) montaniaRusa(v);
3     if (rng(20)) autitosChocadores(v);
4     if (rng(20)) comedor(v);
5     if (rng(20)) teatro(v);
6     if (rng(25)) carreraGomones(v);
7     // ... etc
8 }
```

Probabilidades de visita:

- 20% - Montaña Rusa
- 20% - Autitos Chocadores
- 20% - Comedor
- 20% - Teatro

- 20% - Realidad Virtual
- 10% - Área de Premios (requiere fichas)
- 25% - Carrera de Gómones

3.4 Visitante y Sistema de Fichas

3.4.1 Visitante.java

Responsabilidad: Representar un agente concurrente que navega por el parque.

Atributos:

- ID: BigInteger - Identificador único inmutable.
- ticketNumero: BigInteger - Ticket asignado por el molinete.
- parque: Referencia al coordinador central.
- billetera: Sistema de fichas del visitante.

3.4.2 Billetera.java

Responsabilidad: Gestionar fichas del visitante.

Operaciones:

- cargarFichas(int): Incrementa fichas (acceso controlado por el caller).
- quitarFichas(int): Decrementa si hay suficientes.
- getFichas(): Consulta saldo actual.

Nota importante: La sincronización no está en esta clase, sino en los puntos de intercambio (Ticketera y Área de Premios).

3.4.3 Ticketera.java

Responsabilidad: Hilo que intercambia billeteras para cargar/descontar fichas.

Mecanismo: Exchanger Pattern

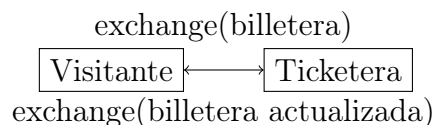


Figure 4: Intercambio sincronizado con Exchanger

Tipos de fichas:

- MR-FICHAS: 3 fichas (Montaña Rusa)
- AC-FICHAS: 2 fichas (Autitos Chocadores)
- AI-FICHAS: 1 ficha (Genérico)
- RV-FICHAS, GOMONES-FICHAS: valores por defecto

Parámetros:

- suma: true = entregar fichas, false = descontar.
- monto: cantidad fija (-1 para usar valorFicha()).

4 Atracciones Mecánicas

4.1 Montaña Rusa

Archivo: parque/juegosMecanicos/MontaniaRusa.java

Responsabilidad: Atracción con capacidad limitada, cola de espera y sistema de viajes por tandas.

Parámetros de configuración:

- CAPACIDAD_COLA: 10 visitantes.
- CAPACIDAD_VIAJE: 5 visitantes por carrito.

Mecanismos de sincronización:

Mecanismo	Capacidad	Propósito
BlockingQueue	10	Cola de espera FIFO con bloqueo
Semaphore (carrito)	5, fair	Control de acceso al carrito
Semaphore (barrier)	0	Barrera para sincronizar inicio de viaje
Semaphore (mutex)	1, fair	Exclusión mutua en obtención de fichas
Exchanger	-	Intercambio de billetera con Ticketera

Table 1: Mecanismos de sincronización en Montaña Rusa

Algoritmo de barrera manual:

```

1 if (carrito.availablePermits() > 0) {
2     // No soy el ltimo , espero
3     barrier.acquire();
4 } else {
5     // Soy el ltimo , hago el viaje
6     Thread.sleep(5000);
7     // Despierto a los dem s
8     barrier.release(CAPACIDAD_VIAJE - 1);
9     // Reinicio el carrito
10    carrito.release(CAPACIDAD_VIAJE);
11 }
```

4.2 Autitos Chocadores

Archivo: parque/juegosMecanicos/AutitosChocadores.java

Responsabilidad: Atracción con capacidad total fija y sincronización mediante CyclicBarrier.

Configuración:

- PERSONAS_POR_AUTO: 2.
- Cantidad de autos: configurable al crear la instancia.
- CAPACIDAD_TOTAL: PERSONAS_POR_AUTO * cantidadAutos.

Mecanismos de sincronización:

- Semaphore autosDisponibles: Control de capacidad.
- CyclicBarrier: Sincroniza inicio cuando todos los lugares están ocupados.
- Semaphore mutex(1): Protege intercambio con Ticketera.

5 Atracciones Complejas

5.1 Carrera de Gomones

Responsabilidad: Atracción multi-etapa con múltiples recursos y coordinación compleja.

Componentes del sistema:

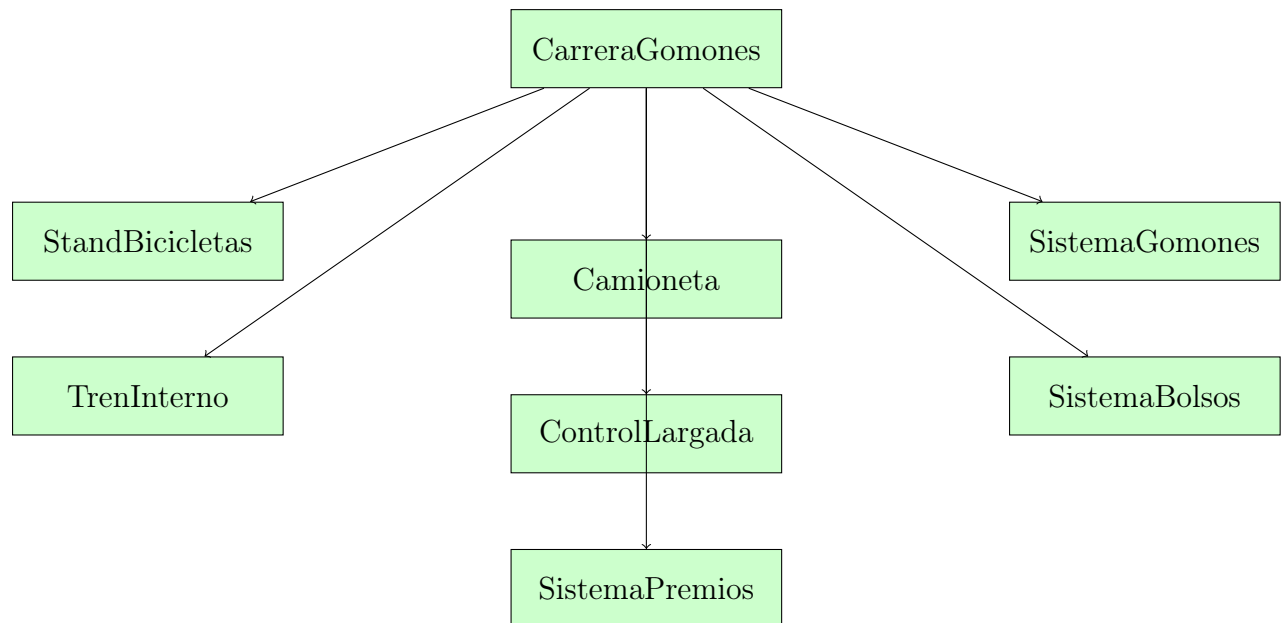


Figure 5: Arquitectura de Carrera de Gomones

5.1.1 Flujo Completo de Participación

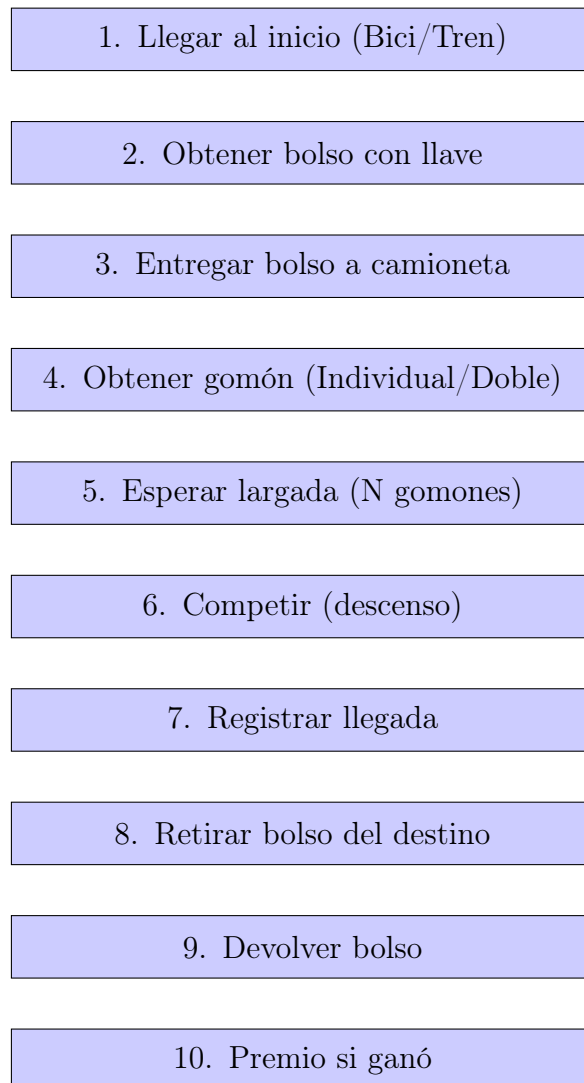


Figure 6: Flujo de participación en la Carrera de Gomones

6 Área de Premios: Diseño y Concurrencia

6.1 Responsabilidad del Componente

El `AreaPremios` funciona como el punto central de canje de fichas por regalos. Es un **recurso compartido y crítico** que gestiona el catálogo de premios y una cola de solicitudes. Su diseño se enfoca en la **concurrencia segura** utilizando el patrón **Productor-Consumidor** de Java.

6.2 Diagrama de Clases Simplificado (UML)

El diseño se compone de tres clases principales que interactúan para gestionar la solicitud de premios, más una clase interna para el control de sincronización.

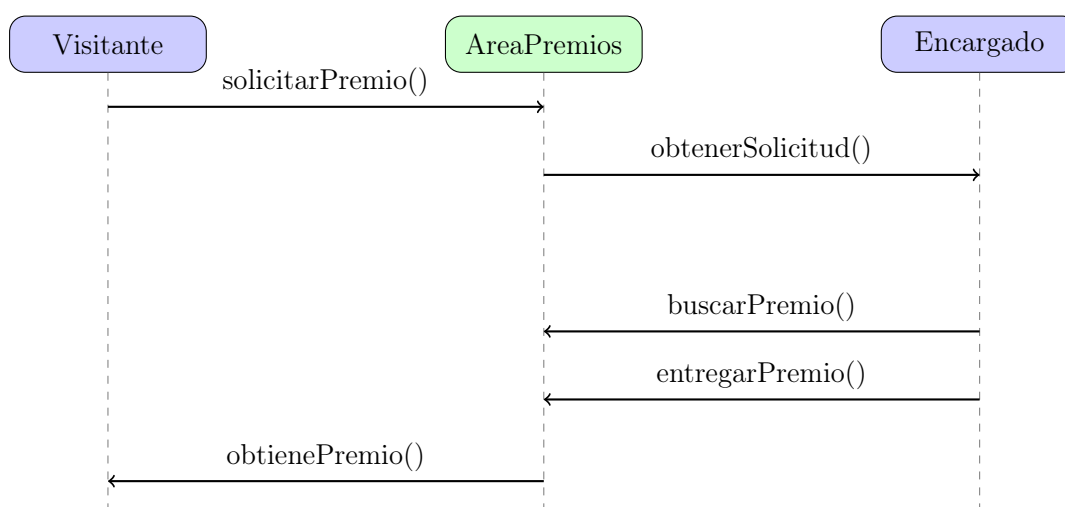


Figure 7: Diagrama de Interacción Productor-Recurso-Consumidor

6.3 Catálogo Inicial de Premios

La lógica de dominio del encargado busca siempre el premio de **mayor costo** que el visitante pueda pagar y que **tenga stock** (Política de Priorización de Gasto).

Table 2: Catálogo de Premios Inicial

Nombre del Premio	Costo (Fichas)	Stock Inicial
Llavero	1	50
Sticker	1	100
Gorra	3	30
Remera	5	25
Peluche pequeño	8	20
Taza	10	15
Mochila	12	10
Peluche grande	15	8
Auriculares	20	5
Reloj	25	3
Consola portátil	30	2

6.4 Modelo de Concurrency y Sincronización

El `AreaPremios` implementa el patrón ****Productor-Consumidor**** utilizando `ReentrantLock` y `Conditions`.

1. `ReentrantLock`: Provee **exclusión mutua** con **fairness**.
2. `Condition hayVisitantes`: Condición global para el **Encargado** (consumidor).
3. `Condition miCondition`: Condición **única por Visitante** (productor), usada para el despertar preciso.

6.5 Lógica de programa

Método `AreaPremios.buscarMejorPremio` Este método implementa la ****Política de Priorización de Gasto****, asegurando que el visitante siempre obtenga el premio de mayor valor que pueda pagar, si hay stock.

```
1 Premio buscarMejorPremio(int fichasDisponibles) {
2     lock.lock();
3     try {
4         Premio mejorPremio = null;
5         // 1. Itera sobre el cat logo
6         for (Premio premio : catalogoPremios) {
7             // 2. Verifica Stock y Costo
8             if (premio.hayStock() && premio.getCostoFichas() <=
9 fichasDisponibles) {
10                 // 3. Prioriza el MAYOR costo
11                 if (mejorPremio == null ||
12                     premio.getCostoFichas() > mejorPremio.getCostoFichas
13 ()) {
14                     mejorPremio = premio;
15                 }
16             }
17         }
18         return mejorPremio;
19     } finally {
20         lock.unlock();
21     }
22 }
```

Listing 1: `AreaPremios.buscarMejorPremio` - Lógica de Priorización

7 Comedor

7.1 Descripción General

El comedor contiene un conjunto fijo de mesas. Cada visitante que desea almorzar solicita una mesa libre; si no hay mesas disponibles, el visitante espera. Una vez que una mesa alcanza su capacidad máxima, todos los visitantes de la mesa comienzan a comer. Cuando todos terminan, la mesa vuelve a quedar disponible y el comedor despierta a los visitantes que estaban esperando lugar.

El sistema garantiza:

- Exclusión mutua en el acceso a las mesas.
- Evitar deadlocks gracias al orden estricto de adquisición de locks.
- Despertar eficiente de hilos bloqueados.

7.2 Clases Principales

Comedor

- Mantiene la lista de mesas.
- Gestiona el lock global que controla el acceso a la búsqueda de mesas disponibles.
- Contiene la condición `hayLugar`, usada para suspender visitantes cuando no existen mesas disponibles.
- Orquesta todo el flujo de almuerzo.

Mesa

- Cada mesa tiene su propio lock para gestionar a sus visitantes.
- Usa la condición `mesaCompleta` para bloquear a los visitantes hasta que la mesa llegue a capacidad máxima.
- Controla el estado interno: visitantes sentados, bandera de “comiendo”, y notificación al comedor cuando la mesa queda libre.

Flujo de interacción entre visitante, comedor y mesa

7.3 Flujo de Ejecución Detallado

1. Llegada del visitante

1. El visitante llama a `almorzar()`.
2. El comedor intenta asignarle una mesa.

2. Búsqueda de una mesa disponible

- El comedor toma su lock global.
- Itera sobre todas las mesas.
- Si una mesa está disponible y acepta al visitante con `intentarSentarse()`, el visitante es asignado.
- Si no existe ninguna mesa disponible, el visitante espera en la condición `hayLugar`.

3. Espera hasta completar la mesa

- Cada visitante llama `esperarYComer()` en su mesa asignada.
- Se bloquea en `mesaCompleta.await()` hasta que la mesa alcance su capacidad.

4. Inicio del almuerzo

- El primer visitante que detecta que la mesa está completa activa el estado `comiendo = true`.
- Se despierta a todos los visitantes de la mesa.

5. Finalización y liberación

- Cada visitante termina de comer y decrementa el contador de ocupación de la mesa.
- El último en abandonar la mesa la marca como libre.
- Se notifica al comedor mediante `notificarLugarDisponible()` para despertar visitantes en espera.

7.4 Mecanismos de Concurrency Utilizados

Locks y Bloqueos

- Se emplea un `ReentrantLock` por mesa.
- Existe un lock global en el comedor para controlar asignaciones.

Condiciones

- `hayLugar`: usada por visitantes cuando no hay mesas disponibles.
- `mesaCompleta`: usada por las mesas para esperar a que la capacidad se complete.

Prevención de Deadlocks

- Nunca se toma el lock de `Comedor` desde dentro del lock de una `Mesa`.
- La notificación al comedor siempre se hace **fuera** del bloqueo de la mesa.

8 Teatro

8.1 Descripción General

El módulo **Teatro** modela un sistema concurrente donde múltiples visitantes intentan acceder a una sala y varios asistentes se encargan de formar grupos de ingreso. El teatro funciona por tandas: cada tanda consta de la formación de 4 grupos de 5 personas. Una vez formados los grupos, comienza el espectáculo; durante este tiempo nadie puede entrar.

Se emplean hilos tanto para los asistentes como para los visitantes, y se utiliza un mecanismo de exclusión mutua mediante **ReentrantLock** y varias condiciones para coordinar estados y puntos de sincronización.

8.2 Clases Principales

- **Teatro**: administra la cola de visitantes, los grupos, el inicio y fin del espectáculo, y coordina toda la sincronización.
- **Asistente**: hilo encargado de formar grupos siempre que haya suficientes visitantes y que el espectáculo no haya iniciado.
- **Visitante**: hilo que intenta entrar al teatro, espera ser habilitado por un asistente y participa del flujo completo del espectáculo.

8.3 Flujo del Sistema

8.3.1 Visitante

1. Intenta entrar; si hay espectáculo en curso, se rechaza.
2. Se encola en el lobby.
3. Espera a ser habilitado por un asistente.
4. Una vez habilitado, se sienta y espera el inicio del espectáculo.
5. Disfruta el espectáculo.
6. Espera el fin y se retira.

8.3.2 Asistente

1. Espera que haya visitantes en cola.
2. Forma grupos de 5 personas.
3. Cuando se formaron 4 grupos, inicia el espectáculo.
4. Tras 10 segundos, finaliza el espectáculo.

8.4 Mecanismos de Sincronización Utilizados

- **ReentrantLock**: asegura exclusión mutua en toda operación crítica del teatro.
- **Condition hayVisitantes**: usada por los asistentes para esperar a que haya suficientes visitantes en la cola.
- **Condition visitantesLiberados**: los visitantes esperan aquí hasta ser seleccionados para un grupo.
- **Condition inicioEspectaculo**: los visitantes esperan hasta que se dé inicio oficial.
- **Condition finEspectaculo**: los visitantes esperan la finalización para retirarse.

8.5 Estados del Teatro

- **esperando_visitantes**: no hay suficientes personas para formar grupo.
- **formando_grupos**: asistentes asignando grupos hasta llegar al máximo.
- **espectaculo_activo**: espectáculo en curso, nadie entra ni se forman grupos.
- **finalizado**: se limpian grupos y se permite una nueva tanda.

9 Realidad Virtual

9.1 Descripción General

La atracción de **Realidad Virtual (VR)** permite a los visitantes disfrutar de una experiencia inmersiva. Cada visitante necesita un equipo completo compuesto por:

- 1 visor de VR
- 2 manoplas
- 1 base

El sistema gestiona la entrega de componentes mediante un **Encargado** que distribuye los elementos disponibles y un registro de visitantes que están esperando o utilizando los equipos.

9.2 Clases Principales

- **RealidadVirtual**: clase principal que coordina la asignación y devolución de equipos, así como la interacción con la ticketera para entregar fichas.
- **Encargado**: hilo encargado de distribuir los componentes de los equipos a los visitantes en la cola.
- **EquipoVR**: representa el equipo de VR de un visitante, indicando qué componentes tiene.
- **Pedido**: objeto que asocia un visitante con su equipo y un *Condition* para esperar a que se complete.

9.3 Variables y Mecanismos de Sincronización

- **visoresDisponibles, manoplasDisponibles, basesDisponibles**: cantidad de componentes disponibles para asignar.
- **cola**: *Queue* de visitantes esperando su equipo completo.
- **equiposEnUso**: *Map* de visitantes que ya tienen su equipo asignado.
- **lock**: *ReentrantLock* para sincronizar acceso a la cola y a los recursos.
- **visitanteEsperando**: *Condition* para que los visitantes esperen hasta que haya componentes disponibles.
- **mutexTicketera**: *Semaphore* que asegura que un visitante acceda a la ticketera a la vez.

9.4 Flujo de Interacción

El flujo básico de interacción es:

1. El visitante llega a la atracción y se crea un **Pedido** con su **EquipoVR**.
2. Se agrega el pedido a la **cola** y se notifica al **Encargado**.
3. El **Encargado** distribuye los componentes disponibles a los visitantes en orden de llegada:
 - Primero se entrega el visor si está disponible.
 - Luego se entregan las manoplas, hasta completar 2.
 - Finalmente se entrega la base si está disponible.
4. Cuando el equipo está completo, el visitante es notificado mediante **Condition.signal()** y puede iniciar la experiencia VR.
5. Tras finalizar, el visitante devuelve el equipo, actualizando los contadores de componentes disponibles y notificando a la cola en espera.
6. Posteriormente, obtiene las fichas de la ticketera mediante un **Exchanger** protegido por un **Semaphore**.

9.5 Diagrama de Flujo de Interacción

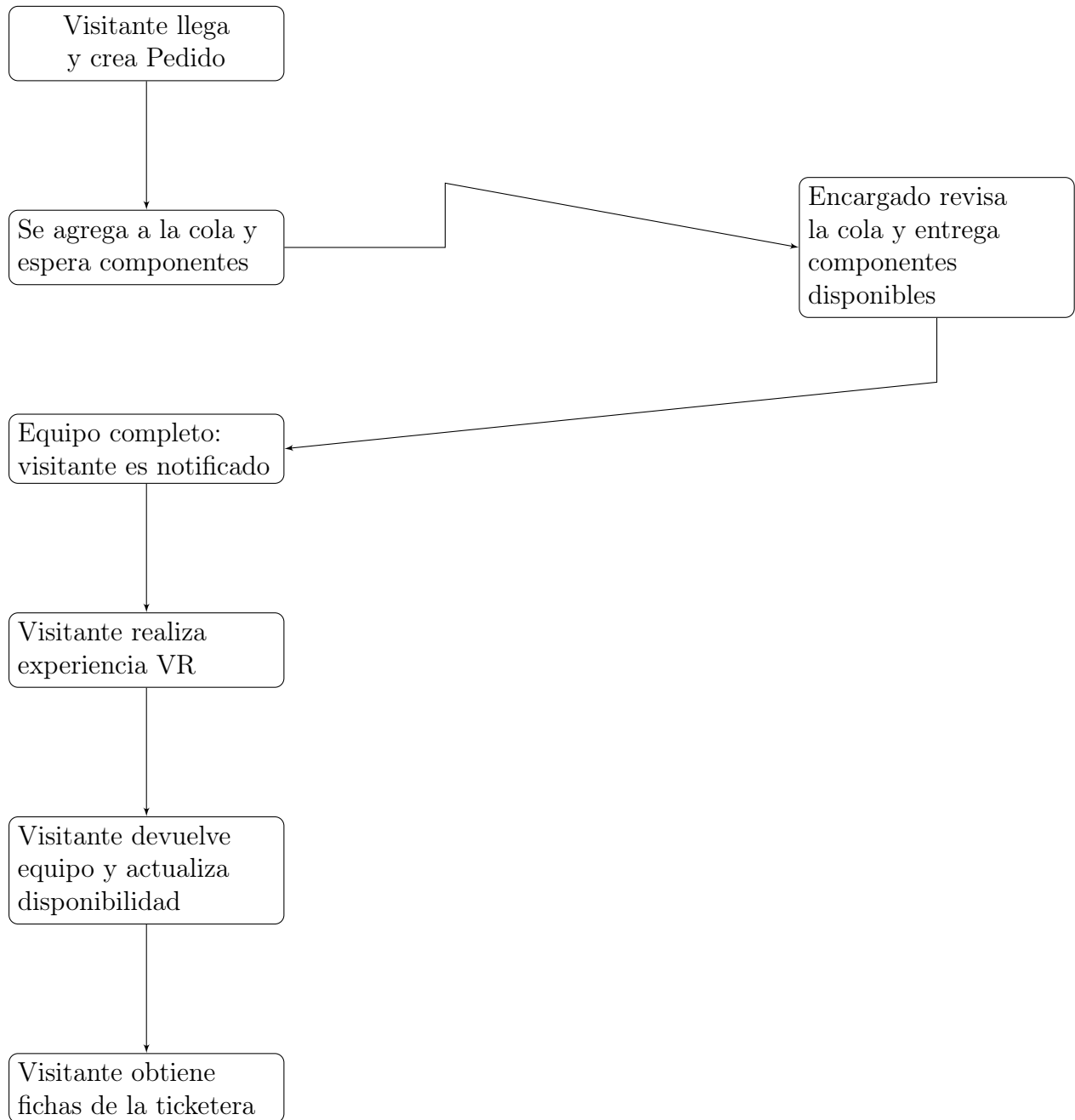


Figure 8: Flujo de interacción entre visitante y encargado en la atracción de Realidad Virtual

9.6 Observaciones

- Se utiliza un **Lock** justo para evitar condiciones de carrera al manipular la cola y los contadores de componentes.
- Cada visitante tiene su propio **Condition** para esperar que su equipo se complete sin bloquear a otros visitantes.

- La ticketera usa un **Semaphore** y un **Exchanger** para manejar la actualización de la billetera del visitante de manera segura.

10 Estrategias de Prevención de Problemas de Concurrencia

10.1 Deadlock

El deadlock ocurre cuando dos o más hilos quedan esperando indefinidamente por recursos que el otro tiene. Para evitarlo, el sistema implementa las siguientes estrategias:

- **Orden estricto de adquisición de locks:** En el Comedor, nunca se adquiere el lock del Comedor desde dentro del lock de una Mesa. La notificación al Comedor se hace fuera del bloqueo de la mesa.
- **Timeouts en bloqueos:** Los Semaphore con `tryAcquire()` permiten abandonar colas de espera saturadas (ej: Montaña Rusa con cola llena).
- **Un solo punto de sincronización por operación:** Cada atracción maneja su propia lógica de sincronización de forma independiente.

10.2 Inanición (Starvation)

La inanición ocurre cuando un hilo nunca obtiene acceso a un recurso compartido. El sistema la previene mediante:

- **Fairness en Semáforos:** Todos los Semaphore se crean con `fair = true`, garantizando que los hilos esperen en orden FIFO.
- **Exhaustividad de recursos:** Las colas de espera tienen límites; cuando están llenas, el visitante puede ir a otra atracción en lugar de esperar indefinidamente.
- **CyclicBarrier:** Garantiza que todos los participantes de un grupo avancen juntos.

10.3 Livelock

El livelock ocurre cuando los hilos están constantemente cambiando de estado pero no progresan. Se evita mediante:

- **Condiciones de espera específicas:** Cada visitante espera en su propia `Condition`, no en un estado compartido activo.
- **No retoma automática de operaciones fallidas:** Si una operación no puede completarse, el hilo sale y puede reintentar más tarde.

11 Tabla Resumen de Mecanismos de Sincronización

Table 3: Mecanismos utilizados por atracción

Atracción	Mecanismo Principal	Propósito
Molinetes	Semaphore(1, fair)	Exclusión mutua en el ingreso
Montaña Rusa	BlockingQueue + Semaphore	Cola de espera y control de capacidad del carrito
Autitos Chocadores	CyclicBarrier	Sincronizar inicio cuando todos los autos están llenos
Realidad Virtual	ReentrantLock + Condition	Asignación de componentes por visitante
Carrera de Gomones	Exchanger + Semaphore	Intercambio de bolsos y control de largada
Área de Premios	ReentrantLock + Condition	Productor-Consumidor con señal individual
Comedor	ReentrantLock + Condition	Espera hasta mesa completa (4 personas)
Teatro	ReentrantLock + Condition	Formación de grupos de 5 personas
Ticketera	Exchanger + Semaphore	Intercambio thread-safe de fichas

12 Conclusiones

El sistema de parque de diversiones implementado demuestra el uso efectivo de múltiples mecanismos de sincronización de Java para resolver problemas de concurrencia reales. Cada atracción presenta desafíos únicos que requieren soluciones específicas:

- **Montaña Rusa:** Manejo de colas con capacidad limitada y sincronización de grupos.
- **Autitos Chocadores:** Coordinación exacta de 20 personas para iniciar la atracción.
- **Realidad Virtual:** Gestión de recursos limitados (componentes) con espera individual.
- **Carrera de Gomones:** Flujo multi-etapa con coordinación de múltiples recursos.
- **Comedor:** Sincronización de inicio colectivo (todos comen juntos).
- **Teatro:** Formación de grupos y coordinación de espectáculo.

La combinación de Semaphores, Locks, Conditions, CyclicBarriers y Exchangers permite un sistema robusto que evita deadlocks, inanición y livelocks, garantizando que todos los visitantes puedan disfrutar de las atracciones de manera segura y justa.